CS 4032 – Web Programming

National University
Of Computer and Emerging Sciences

Intro to Node.js

CSE 154 Modules

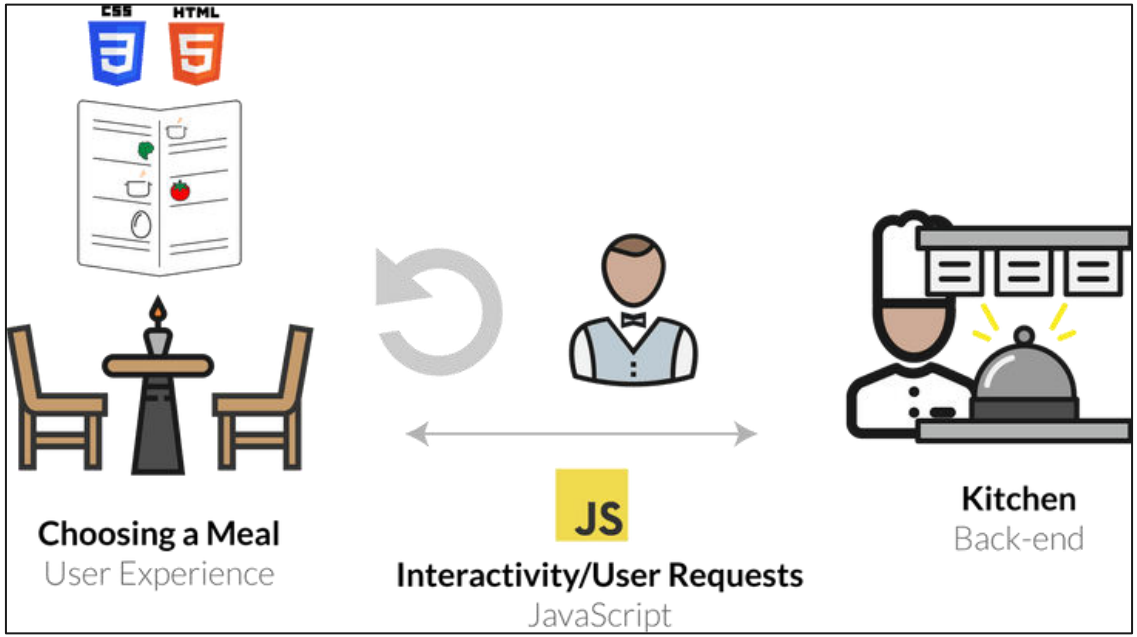
1. Web page structure and appearance with HTML5 and CSS.
2. Client-side interactivity with JS DOM and events.
3. Using web services (APIs) as a client with JS.
4. ***Writing JSON-based web services with a server-side language.***
5. Storing and retrieving information in a database with Mongodb and server-side programs.

Review: Web Service

Web service: software functionality that can be invoked through the internet using common protocols. Done by contacting a program on a web server

- Web services can be written in a variety of languages
- Many web services accept parameters and produce results
- Clients contact the server through the browser using XML over HTTP and/or AJAX Fetch code
- The service's output might be HTML but could be text, XML, JSON, or other content

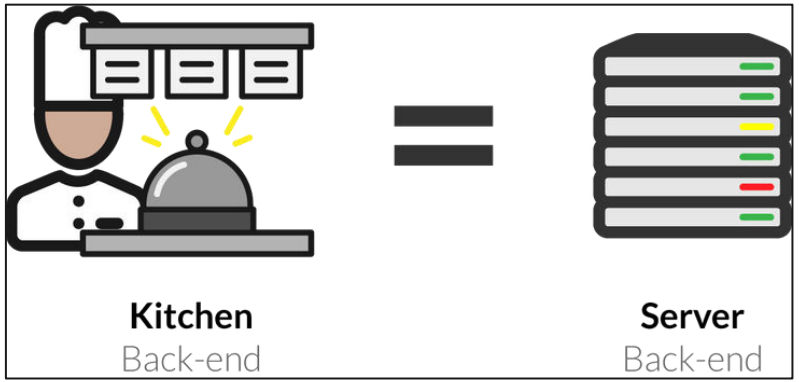
So How Does a Web Service Respond to Requests



Why Do We Need a Server to Handle Web Service Requests?

Servers are dedicated computers for processing data efficiently and delegating requests sent from many clients (often at once).

These tasks are not possible (or appropriate) in the client's browser.



Our (New) Server-Side Language: JS (Node)

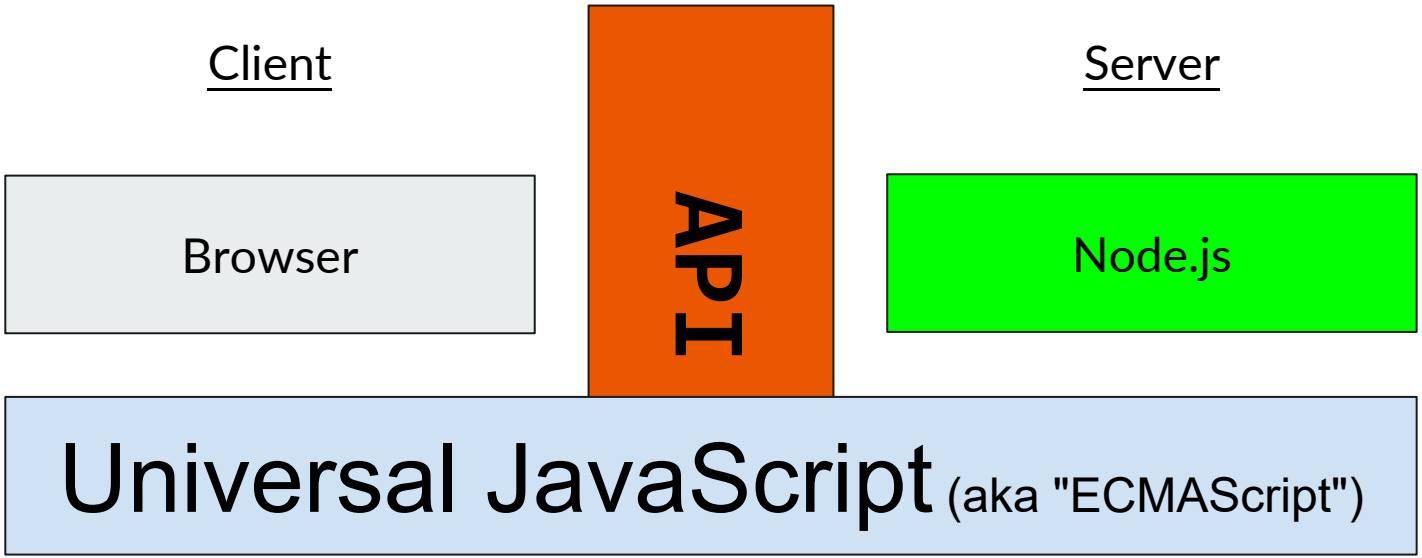
- Open-source with an active developer community
- Flourishing package ecosystem
- Designed for efficient, asynchronous server-side programming
- You can explore other server-side languages after this course!

Languages for Server-Side Programming

Server-side programs are written using programming languages/frameworks such as PHP, Java/JSP, Ruby on Rails, ASP.NET, Python, Perl, JS (Node.js), and many, many more

Web servers contain software to run those programs and send back their output.

Client-side vs. Server-side JavaScript



What is Client-Side JS?

- So far, we have used JS on the browser (client) to add interactivity to our web pages
- "Under the hood", your browser requests the JS (and other files) from a URL resource, loads the text file of the JS, and interprets it realtime in order to define how the web page behaves.
- In Chrome, it does this using the V8 JavaScript engine, which is an open-source JS interpreter made by Google. Other browsers have different JS engines (e.g. Firefox uses SpiderMonkey).
- Besides the standard JS language features, you also have access to the DOM when running JS on the browser - this includes the `window` and `document`

Node.js: Server-side JS

- Node.js uses the same open-source V8 JavaScript engine as Chrome
- Node.js is a runtime environment for running JS programs using the same core language features, but outside of the browser.

- **When using Node, you do not have access to the browser objects/functions (e.g. document, window, addEventListener, DOM nodes).**

- Instead, you have access to functionality for managing HTTP requests, file i/o, and database interaction.
- This functionality is key to building REST APIs!

Client-side vs. Server-side JavaScript

Client-side

- Adheres (mostly) to the ECMAScript standards
- Parsed and run by a browser and therefore has access to `document`, `window`, and more
- Calls APIs using `fetch`, which sends HTTP requests to a server
- Runs only while the web page is open
- Handles "events" like users clicking on a button

Server-side

- Adheres (mostly) to the ECMAScript standards
- Parsed and run by Node.js and therefore has access to File I/O, databases, and more
- Handles requests from clients, sending HTTP responses back
- Runs as long as Node is running and listening
- Handles HTTP requests (GET/POST etc)

What is NodeJS?

- A JavaScript runtime environment running Google Chrome’s V8 engine
  - a.k.a. a server-side solution for JS
  - Compiles JS, making it really fast
- Runs over the command line
- Designed for high concurrency
- Without threads or new processes
  - Never blocks, not even for I/O
- Uses the CommonJS framework
  - Making it a little closer to a real OO language

Concurrency: The Event Loop

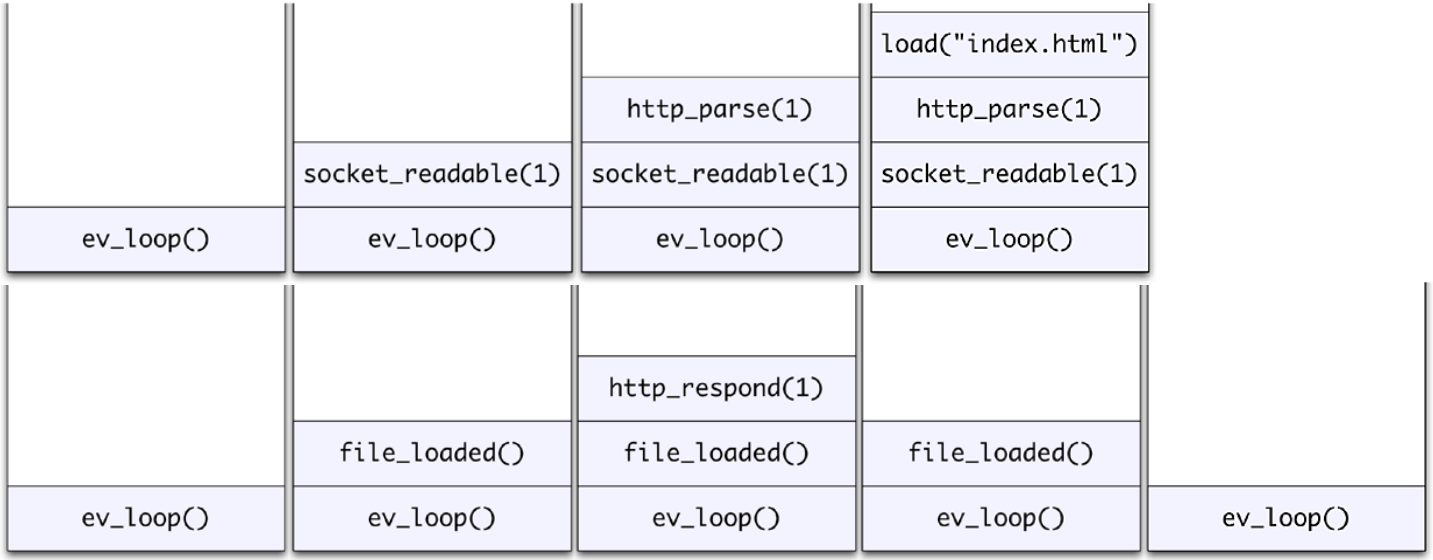
- Instead of threads Node uses an event loop with a stack
- Alleviates overhead of context switching

Threads VS Event-driven

Threads	Asynchronous Event-driven
Lock application / request with listener-workers threads	only one thread, which repeatedly fetches an event
Using incoming-request model	Using queue and then processes it
multithreaded server might block the request which might involve multiple events	manually saves state and then goes on to process the next event
Using context switching	no contention and no context switches
Using multithreading environments where listener and workers threads are used frequently to take an incoming-request lock	Using asynchronous I/O facilities (callbacks, not poll/select or O_NONBLOCK) environments

Event Loop Example

- Request for “index.html” comes in
- Stack unwinds and ev_loop goes to sleep
- File loads from disk and is sent to the client



Non-blocking I/O

- Servers do nothing but I/O
    - Scripts waiting on I/O requests degrades performance
- To avoid blocking, Node makes use of the event driven nature of JS by attaching callbacks to I/O requests
- Scripts waiting on I/O waste no space because they get popped off the stack when their non-I/O related code finishes executing

Consistency

- Use of JS on both the client and server-side should remove need to “context switch”
  - Client-side JS makes heavy use of the DOM, no access to files/databases
  - Server-side JS deals mostly in files/databases, no DOM
    - JSDom project for Node works for simple tasks, but not much else

Getting started with Node.js

When you have Node installed, you can run it immediately in the command line.

1. Start an interactive REPL with `node` (no arguments). This REPL is much like the Chrome browser's JS console tab.
    a. ("REPL" stands for "Read Evaluate Print Loop". It's a kind of tool that allows you to run one line of code at a time. REPL-style tools exist in most languages.)
2. Execute a JS program in the current directory with `node file.js`

Starting a Node.js Project

There are a few steps to starting a Node.js application, but luckily most projects will follow the same structure.

When using Node.js, you will mostly be using the command line

1. Start a new project directory (e.g. `node-practice`)
2. Inside the directory, run `npm init` to initialize a `package.json` configuration file (you can keep pressing Enter to use defaults)
3. Install any modules with `npm install <package-name>`
4. Write your Node.js file! (e.g. `app.js`)
5. Include any front-end files in a `public` directory within the project.

Along the way, a tool called `npm` will help install and manage packages that are useful in your Node app.

Starting a Node.js Project

Run `npm init` to create `package.json`

Starting a Node.js Project

Run `npm install express` to install the Express.js package

Starting `app.js`

Build your basic app:

```
"use strict";

const express = require('express');
const app = express();

app.get('/posts', function (req, res) {
  res.type("text").send("Hello World");
});

app.listen(8080);
```

Node.js Modules

When you run a `.js` file using Node.js, you have access to default functions in JS (e.g. `console.log`)

In order to get functionality like file i/o or handling network requests, you need to import that functionality from modules - this is similar to the `import` keyword you have used in Java or Python.

In Node.js, you do this by using the `require()` function, passing the string name of the module you want to import.

For example, the module we'll use to respond to HTTP requests in Node.js is called `express`. You can import it like this:

```
const express = require("express");
```

Quick reminder on `const` Keyword

Using `const` to declare a variable inside of JS just means that you can never change what that variable references. We've used this to represent "program constants" indicated by ALL_UPPERCASE naming conventions
For example, the following code would not work:

```
const specialNumber = 1;
specialNumber = 2; // TypeError: Assignment to constant variable.
```

When we store modules in Node programs, it is conventional to use `const` instead of `let` to avoid accidentally overwriting the module.
Unlike the program constants we define with `const` (e.g. `BASE_URL`), we use camelCase naming instead of ALL_CAPS.

app.listen()

To start the localhost server to run your Express app, you need to specify a port to listen to.

The express app object has a function `app.listen` which takes a port number and optional callback function

At the bottom of your `app.js`, add the following code - (`process.env.PORT` is needed to use the default port when hosted on an actual server)

```
// Allows us to change the port easily by setting an environment
// variable, so your app works with our grading software
const PORT = process.env.PORT || 8000;
app.listen(PORT);
```

Basic Routing in Express

Routes are used to define endpoints in your web service
Express supports different HTTP requests - we will learn GET and POST
Express will try to match routes in the order they are defined in your code

Adding Routes in Express.js

```
app.get(path, (req, res) => {
  ...
});
```

- `app.get` allows us to create a GET endpoint. It takes two arguments: The endpoint URL path, and a callback function for modifying/sending the response.
- `req` is the request object, and holds items like the request parameters.
- `res` is the response object, and has methods to send data to the client.
- `res.set(...)` sets header data, like "content-type". Always set either "text/plain" or "application/json" with your response.
- `res.send(response)` returns the response as HTML text to the client.
- `res.json(response)` Does the same, but with a JSON object.

When adding a route to the path, you will retrieve information from the request, and send back a response using res (e.g. setting the status code, content-type, etc.)

If the visited endpoint has no matching route in your Express app, the response will be a 404 (resource not found)

Useful Request Properties/Methods

Name	Description
`req.params`	Endpoint “path” parameters from the request
`req.query`	Query parameters from the request

Useful Response Properties/Methods

Name	Description
`res.write(data)`	Writes data in the response without ending the communication
`res.send()`	Sends information back (default text with HTML content type)
`res.json()`	Sends information back as JSON content type
`res.set()`	Sets header information, such as "Content-type"
`res.type()`	A convenience function to set content type (use "text" for "text/plain", use "json" for "application/json")
`res.status()`	Sets the response status code
`res.sendStatus()`	Sets the response status code with the default status text

Setting the Content Type

By default, the content type of a response is HTML - we will only be sending plain text or JSON responses though in our web services

To change the content type, you can use the `res.set` function, which is used to set response header information (e.g. content type).

You can alternatively uses `res.type("text")` and `res.type("json")` which are equivalent to setting `text/plain` and `application/json` Content-Type headers, respectively.

```
app.get('/hello', function (req, res) {
  // res.set("Content-Type", "text/plain");
  res.type("text"); // same as above
  res.send('Hello World!');
});
```

```
app.get('/hello', function (req, res) {
  // res.set("Content-Type", "application/json");
  res.json({ "msg" : "Hello world!" });
  // can also do res.json({ "msg" : "Hello world!"});
  // which also sets the content type to application/json
});
```

Request Parameters: Path Parameters

Act as wildcards in routes, letting a user pass in "variables" to an endpoint

Define a route parameter with `:param`

```
Route path: /states/:state/cities/:city
Request URL: http://localhost:8000/states/wa/cities/Seattle
req.params: { "state": "wa", "city": "Seattle" }
```

These are attached to the request object and can be accessed with `req.params`

```
app.get("/states/:state/cities/:city", function (req, res) {
  res.type("text");
  res.send("You sent a request for " + req.params.city + ", "
    + req.params.state);
});
```

Request Parameters: Path Parameters

You can also use query parameters in Express using the `req.query` object, though they are more useful for optional parameters.

```
Route path: /cityInfo
Request URL: http://localhost:8000/cityInfo?state=wa&city=Seattle
req.query: { "state": "wa", "city": "Seattle" }
```

```
app.get("/cityInfo", function (req, res) {
  let state = req.query.state; // wa
  let city = req.query.city;   // Seattle
  // do something with variables in the response
});
```

Unlike path parameters, these are not included in the path string (which are matched using Express routes) and we can't be certain that the accessed query key exists.
If the route requires the parameter but is missing, you should send an error to the client in the response.

Step by Step -Tutorial

- Follow the below link and create backend side only.
- https://www.positronx.io/react-mern-stack-crud-app-tutorial/